

BUILDING A PREDICTIVE MODEL FOR CUSTOMER CHURN FOR SYRIATEL

Student Name: Stephen Jilani Project: Phase 3 Date: 6 May 2026

1. BUSINESS UNDERSTANDING

SyriaTel has been having a challenge of customer churn, and view customer retention as a better move compared to attracting and onboarding new clients. SyriaTel therefore, has an objective of building a predictive model to help identify customers likely to churn and switch to competitors.

STAKEHOLDERS

- The possible stakeholders for the project are as below:
 - a. Customer retention teams.
 - b. Marketing and business development teams.
 - c. Business decision makers at SyriaTel.

BUSINESS OBJECTIVES The primary objective is to ascertain if there are any predictable trends in the data that can help the company make informed decisions. The specific objectives are as follows

1. To identify any patterns indicated by the data as far as customers leaving the company is concerned.
2. Build a predictive model that predicts whether a customer is expected to churn.
3. To provide recommendations on what can be done differently to ensure customer retention.

BUSINESS QUESTIONS

1. Are there any trends in the data presented?
2. Are the trends predictive of actual customer churn?

The project is expected to be a success if it ends up with a model with the ability to correctly identify customers who are likely to churn.

2. DATA UNDERSTANDING

The data was obtained from Kaggle and represents the Churn in SyriaTel's customers. This is the rate at which customers leave the company and end up procuring the services of competitors. The dataset contains data about customers ranging from the location of a customer, account numbers, minutes of calls across different service packages i.e local calls, international calls etc., telephone charges, customer churn etc.

Preliminary analysis on the data for better understanding and determining the best metrics that will achieve the objectives of the project

```
#Importation of necessary libraries for the project.
import pandas as pd
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from sklearn.linear_model import Lasso, Ridge, LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, PolynomialFeatures,
StandardScaler
```

#Loading the dataset

```
df = pd.read_csv('bigml_59c28831336c6604c800002a.csv')
df.head()
```

	state	account length	area code	phone number	international plan	\
0	KS	128	415	382-4657	no	
1	OH	107	415	371-7191	no	
2	NJ	137	415	358-1921	no	
3	OH	84	408	375-9999	yes	
4	OK	75	415	330-6626	yes	

	voice mail plan	number vmail messages	total day minutes	total day calls	\
0	yes	25	265.1	110	
1	yes	26	161.6	123	
2	no	0	243.4	114	
3	no	0	299.4	71	
4	no	0	166.7	113	

	total day charge	...	total eve calls	total eve charge	\
0	45.07	...	99	16.78	
1	27.47	...	103	16.62	
2	41.38	...	110	10.30	
3	50.90	...	88	5.26	
4	28.34	...	122	12.61	

	total night minutes	total night calls	total night charge	\
0	244.7	91	11.01	
1	254.4	103	11.45	
2	162.6	104	7.32	
3	196.9	89	8.86	
4	186.9	121	8.41	

	total intl minutes	total intl calls	total intl charge	\
0	10.0	3	2.70	
1	13.7	3	3.70	
2	12.2	5	3.29	
3	6.6	7	1.78	

4	10.1	3	2.73
---	------	---	------

	customer service calls	churn
0	1	False
1	1	False
2	0	False
3	2	False
4	3	False

[5 rows x 21 columns]

#Checking for the descriptive statistics for the data.

df.info()

#Observations

#The dataset contains 21 columns and 3,333 rows of data, which encompass various aspects of SyriaTel's clients. From phone numbers, to details about the

#calls and packages they subscribe to and whether they have left the company or not i.e churn.

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 3333 entries, 0 to 3332

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	state	3333 non-null	object
1	account length	3333 non-null	int64
2	area code	3333 non-null	int64
3	phone number	3333 non-null	object
4	international plan	3333 non-null	object
5	voice mail plan	3333 non-null	object
6	number vmail messages	3333 non-null	int64
7	total day minutes	3333 non-null	float64
8	total day calls	3333 non-null	int64
9	total day charge	3333 non-null	float64
10	total eve minutes	3333 non-null	float64
11	total eve calls	3333 non-null	int64
12	total eve charge	3333 non-null	float64
13	total night minutes	3333 non-null	float64
14	total night calls	3333 non-null	int64
15	total night charge	3333 non-null	float64
16	total intl minutes	3333 non-null	float64
17	total intl calls	3333 non-null	int64
18	total intl charge	3333 non-null	float64
19	customer service calls	3333 non-null	int64
20	churn	3333 non-null	bool

dtypes: bool(1), float64(8), int64(8), object(4)

memory usage: 524.2+ KB

```

#Checking through the 'churn' column to see the elements contained in
there as the data is in boolean form.
print(df['churn'].unique())
print(df['churn'].value_counts())
#Observation
#The data in the column is in True or False, indicating whether a
client has left the company or not.
#There are 483 customers that have 'True' return in the dataset
indicating that they left the company. This represents around 14% of
the total data which
#is not alarming, however, the company would still be interested in
bringing this figure down.

[False  True]
churn
False    2850
True      483
Name: count, dtype: int64

```

3. DATA PREPARATION AND CLEANING

```

#Identifying blanks rows in the dataset
df.isna().sum()

#Observation
#The data has no missing information signifying that it should be very
clean and fit for the project.

state                0
account length       0
area code            0
phone number         0
international plan   0
voice mail plan      0
number vmail messages 0
total day minutes    0
total day calls      0
total day charge     0
total eve minutes    0
total eve calls      0
total eve charge     0
total night minutes  0
total night calls    0
total night charge   0
total intl minutes   0
total intl calls     0
total intl charge    0
customer service calls 0

```

```

churn                                0
dtype: int64

#Identifying if there are any duplicates in the phone number column,
as that should be the unique identifier
df['phone number'].duplicated().sum()
#The phone number column has no duplicates, meaning that each phone
number in the dataset is unique.

0

```

An extra step could have been renaming the columns. However, there is already consistency in how the columns are named hence renaming columns by adding maybe underscores to separate each word doesn't add any value to the dataset.

```

#Checking the states that are there in the dataset, this is to see if
there is any relationship between churn and the state later
#in the project
df['state'].unique()

array(['KS', 'OH', 'NJ', 'OK', 'AL', 'MA', 'MO', 'LA', 'WV', 'IN',
      'RI',
      'IA', 'MT', 'NY', 'ID', 'VT', 'VA', 'TX', 'FL', 'CO', 'AZ',
      'SC',
      'NE', 'WY', 'HI', 'IL', 'NH', 'GA', 'AK', 'MD', 'AR', 'WI',
      'OR',
      'MI', 'DE', 'UT', 'CA', 'MN', 'SD', 'NC', 'WA', 'NM', 'NV',
      'DC',
      'KY', 'ME', 'MS', 'TN', 'PA', 'CT', 'ND'], dtype=object)

#Running the dataset's columns to identify the columns that are
relevant in the project; and whether there is possibility of creating
secondary datasets
df.columns

Index(['state', 'account length', 'area code', 'phone number',
      'international plan', 'voice mail plan', 'number vmail
      messages',
      'total day minutes', 'total day calls', 'total day charge',
      'total eve minutes', 'total eve calls', 'total eve charge',
      'total night minutes', 'total night calls', 'total night
      charge',
      'total intl minutes', 'total intl calls', 'total intl charge',
      'customer service calls', 'churn'],
      dtype='object')

```

At this point, the data has no duplicates, no blank rows, the columns are also named consistently, the data is also in formats that can be used for further analysis as they are. More columns may need to be added to the dataset to provide further insights and provide different perspectives on why or what customers churn more than others. Columns that are to be

dropped will be dropped after we have done visualization between different aspects of the data to see if there is any relationship with customer churn.

```
#Running the dataset for visibility
```

```
df.head(1)
```

	state	account length	area code	phone number	international plan	\
0	KS	128	415	382-4657		no

	voice mail plan	number vmail messages	total day minutes	total day calls	\
0	yes	25	265.1		

	total day charge	...	total eve calls	total eve charge	\
0	45.07	...	99	16.78	

	total night minutes	total night calls	total night charge	\
0	244.7	91	11.01	

	total intl minutes	total intl calls	total intl charge	\
0	10.0	3	2.7	

	customer service calls	churn
0	1	False

```
[1 rows x 21 columns]
```

```
#Adding a column for total number of minutes made by a customer. This is by adding day minutes, evening minutes, nigh minutes and international minutes
```

```
#The intention is to finally compute an average charge per minute to see if there's consistency in how customers are charged.
```

```
df['total minutes'] = df['total day minutes'] + df['total eve minutes'] + df['total night minutes'] + df['total intl minutes']
```

```
#Running the column to see if changes have been reflected
```

```
df['total minutes'].head()
```

```
0    717.2
1    625.2
2    539.4
3    564.8
4    512.0
```

```
Name: total minutes, dtype: float64
```

```
#Adding a column for total charge by adding all charges in the original dataset. The aim is as mentioned above that we are trying to see if there
```

```
#a patern in how customers are charged.
```

```
df['total charge'] = df['total day charge'] + df['total eve charge'] +
```

```
df['total night charge'] + df['total intl charge']
df['total charge'].head()
```

```
0    75.56
1    59.24
2    62.29
3    66.80
4    52.09
```

```
Name: total charge, dtype: float64
```

#Adding a column on average charge per minute to see if there is consistency on how customers are charged

```
df['average charge'] = df['total charge']/df['total minutes']
df['average charge'].head()
```

```
0    0.105354
1    0.094754
2    0.115480
3    0.118272
4    0.101738
```

```
Name: average charge, dtype: float64
```

#Finding the range of the average charges

```
range_charge = df['average charge'].max() - df['average charge'].min()
range_charge
```

#Observation

#There is already a range of 0.063 per minute which already signifies that there is some inconsistencies on how customers are charged for services. This may

#be a sign of presence of outliers in the data.

#This may be a contribution to customer churn but that will be ascertained as we go.

```
0.06284112317792222
```

#This is to cement the observation above about having inconsistencies in the way customers are charged for services.

```
df['average charge'].describe()
```

#Observations

#The mean, min, max, the percentiles observed below shows that there is some dispersion in the way customers are charged by the company.

```
count    3333.000000
mean      0.100354
std       0.008440
min       0.066950
25%       0.094893
50%       0.100385
75%       0.106056
```

```
max          0.129791
Name: average charge, dtype: float64
```

#Running the dataset for visibility of the columns.

```
df.head()
```

	state	account length	area code	phone number	international plan	\
0	KS	128	415	382-4657	no	
1	OH	107	415	371-7191	no	
2	NJ	137	415	358-1921	no	
3	OH	84	408	375-9999	yes	
4	OK	75	415	330-6626	yes	

	voice mail plan	number vmail messages	total day minutes	total day calls	\
0	yes	25	265.1		
110					
1	yes	26	161.6		
123					
2	no	0	243.4		
114					
3	no	0	299.4		
71					
4	no	0	166.7		
113					

	total day charge	...	total night calls	total night charge	\
0	45.07	...	91	11.01	
1	27.47	...	103	11.45	
2	41.38	...	104	7.32	
3	50.90	...	89	8.86	
4	28.34	...	121	8.41	

	total intl minutes	total intl calls	total intl charge	\
0	10.0	3	2.70	
1	13.7	3	3.70	
2	12.2	5	3.29	
3	6.6	7	1.78	
4	10.1	3	2.73	

	customer service calls	churn	total minutes	total charge	average charge
0	1	False	717.2	75.56	
0.105354					
1	1	False	625.2	59.24	
0.094754					
2	0	False	539.4	62.29	
0.115480					
3	2	False	564.8	66.80	
0.118272					

4	3	False	512.0	52.09
0.101738				

[5 rows x 24 columns]

VISUALIZATION

We visualized different columns and aspects of the raw data to see if there is any correlation with customer churn. This is to determine patterns which will determine the columns and aspects of the data that are relevant to our project.

```
df['churn'].head()
0    False
1    False
2    False
3    False
4    False
Name: churn, dtype: bool
```

SCATTER PLOT FOR NO. OF CUSTOMER SERVICE CALLS AGAINST CHURN

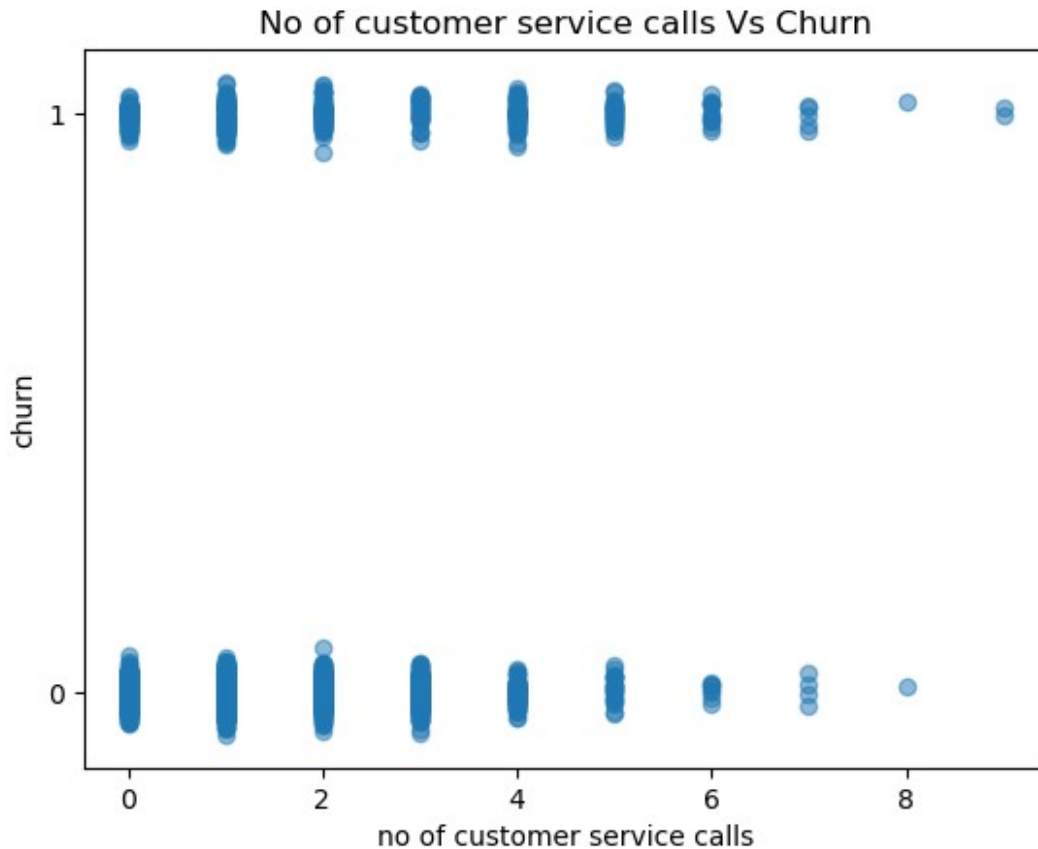
This was chosen to see if there is any sort of relationship between the number of service calls to the churn rate of customers. The assumption before the visualization is that the more the number of customer service calls, the higher the churn rate as it may signify that customers are not satisfied with the services of the company.

```
#Plotting a scatter plot for the number of customer service calls and churn count.
# X = independent variable which is the number of customer service calls

X = df['customer service calls']

#y = dependent variable which is whether there is churn
df['churn']=df['churn'].astype(int)
y = df['churn']

y_jittered = y + np.random.normal(0, 0.02, size=len(y))
plt.scatter(X, y_jittered, alpha=0.5)
plt.yticks([0, 1])
plt.xlabel("no of customer service calls")
plt.ylabel("churn")
plt.title("No of customer service calls Vs Churn")
plt.show()
```



The plot above shows a relationship between customer service calls and churn rate. There are clients that end up churning even where there are zero calls to customer service probably due to different reasons other than customer service calls. However, as the number of calls to customer service increase, the churn rate fluctuates up and down until a point where it reduces. This shows that as much as clients leave the company due to high customer service calls, most of them don't stay to a point where the recurring calls have exceeded 5 leading to the churn rate reducing as the calls to customer service increase.

HISTOGRAM FOR AVERAGE CHARGES PER MINUTE AGAINST CHURN RATE Based on the distribution of the average charges seen above; that there are possible outliers and require a wide range in average charges, there is reason to believe that there is no consistency on how customers are charged by the company.

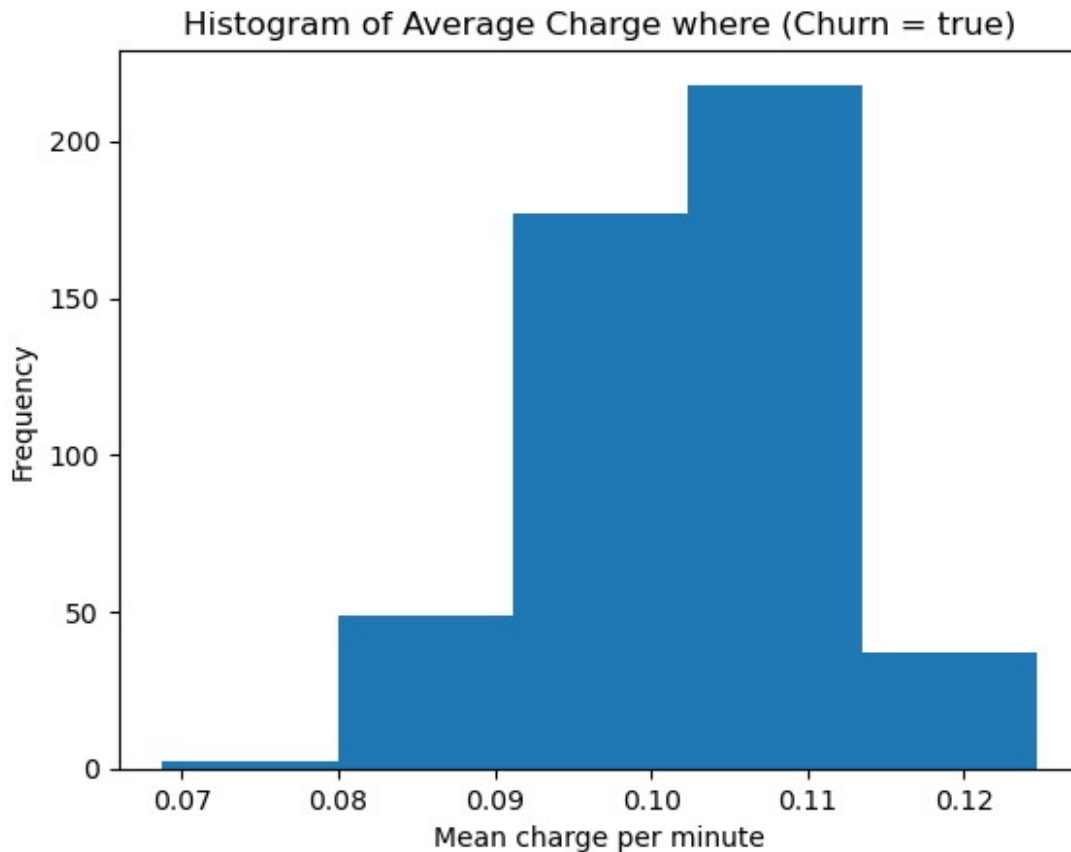
This histogram is meant to disclose if there is any relationship between the amount of churn customers are the average charge per minute goes up.

#Plotting a histogram for the average charge per minute against the frequency of Trues in the 'churn' column

#Filtering the rows where churn is 1 i.e True
`churn_true = df[df['churn'] == 1]`

`plt.hist(churn_true['average charge'], bins= 5)`
`plt.xlabel('Mean charge per minute')`

```
plt.ylabel('Frequency')
plt.title('Histogram of Average Charge where (Churn = true)')
plt.show()
```



The histogram above shows 2 things:

1. That as the average charge per minute increases, there is higher customer churn.
2. That there is inconsistency in how customers are charged in the company and that may be the reason for the alarming churn rate in the company.

#Running the initial dataset for visibility

```
df.head()
```

	state	account length	area	code	phone number	international	plan	\
0	KS	128		415	382-4657		no	
1	OH	107		415	371-7191		no	
2	NJ	137		415	358-1921		no	
3	OH	84		408	375-9999		yes	
4	OK	75		415	330-6626		yes	

	voice mail plan	number vmail	messages	total day minutes	total day
calls \					
0	yes		25	265.1	

```

110
1          yes          26          161.6
123
2          no           0          243.4
114
3          no           0          299.4
71
4          no           0          166.7
113

    total day charge    ...    total night calls    total night charge \
0          45.07    ...          91          11.01
1          27.47    ...         103          11.45
2          41.38    ...         104           7.32
3          50.90    ...           89           8.86
4          28.34    ...         121           8.41

    total intl minutes    total intl calls    total intl charge \
0           10.0           3           2.70
1           13.7           3           3.70
2           12.2           5           3.29
3           6.6           7           1.78
4           10.1           3           2.73

    customer service calls    churn    total minutes    total charge    average
charge
0          1          0          717.2          75.56
0.105354
1          1          0          625.2          59.24
0.094754
2          0          0          539.4          62.29
0.115480
3          2          0          564.8          66.80
0.118272
4          3          0          512.0          52.09
0.101738

[5 rows x 24 columns]

```

BOX PLOT FOR TOTAL MINUTES AGAINST CHURN The expectation is that the customers tat have the most minutes tend to have experienced the most challenges with the company and therefore end up churning. The box plot below was meant to see if there is a relationship between the total minutes and churn rates.

```

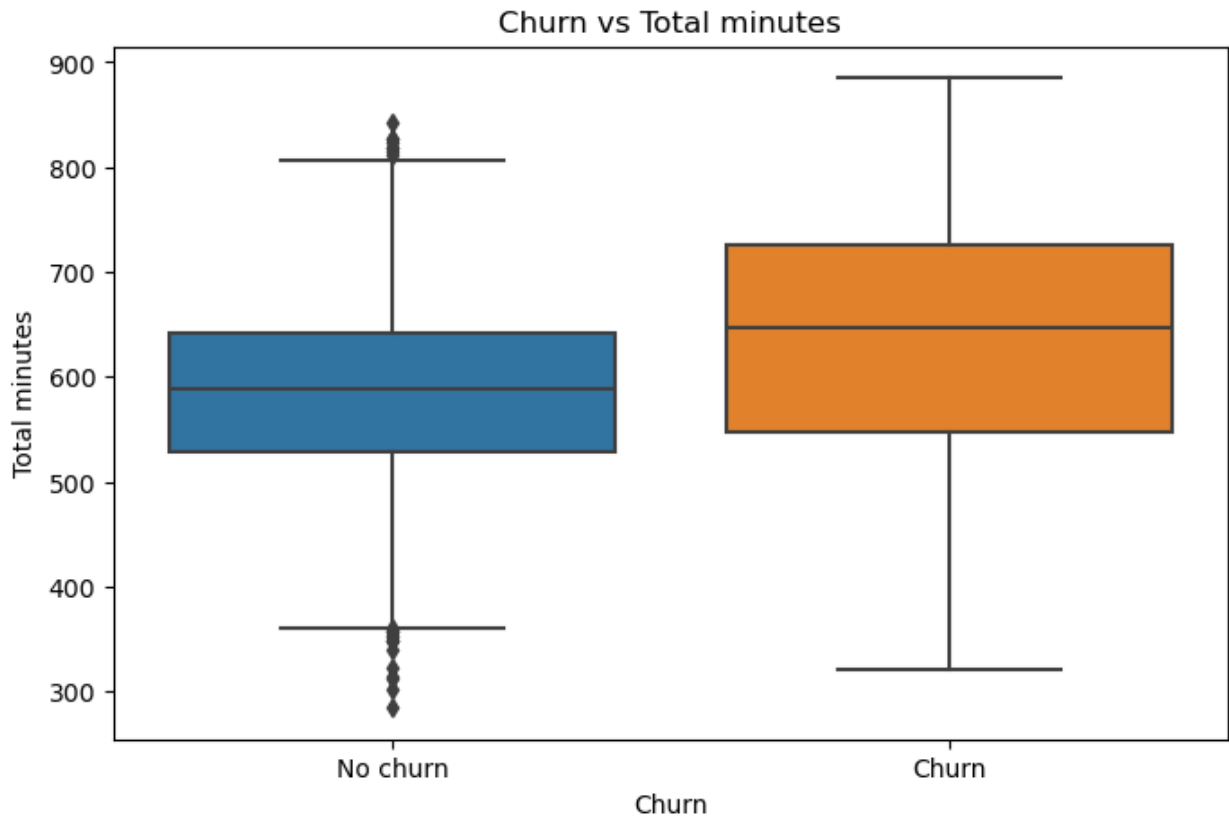
#Plotting box plots to determine whether higher usage customers churn
more or less. This is by using the total minutes.

churn_label = df['churn'].map({0: 'No churn', 1: 'Churn'})
plt.figure(figsize=(8,5))

```

```
sns.boxplot(
    x= churn_label,
    y= df['total minutes']
)

plt.title('Churn vs Total minutes')
plt.xlabel('Churn')
plt.ylabel('Total minutes')
plt.show()
```



The box plot shows that the median of the churn customers is higher than that of the no churn customers. This may be a suggestion that the heavy users are the ones that end up leaving the company probably due to lack of satisfaction, heavy bills or even higher service constraints.

There is also a bigger spread in the churn customers indicating that there is some randomness in the way customers churn in relation to total minutes. There are also more outliers in the churn clients which suggests possibility of other factors that lead to customer churn.

We will plot other visuals below to stress on the relationship between heavy customers and the metrics mentioned above i.e heavy bills, service constraints.

PLOTTING LINE PLOTS BETWEEN CUSTOMER SERVICE CALLS AND TOTAL MINUTES The intention is to see if there is any linear relationship between the number of minutes that a customer has had to the number of customer service calls so that we can see if the long lasting clients have had more challenges with the services of the company.

```
#Plotting a line graph between customer service calls against total minutes to see if the charges go higher as the number of minutes increase.
```

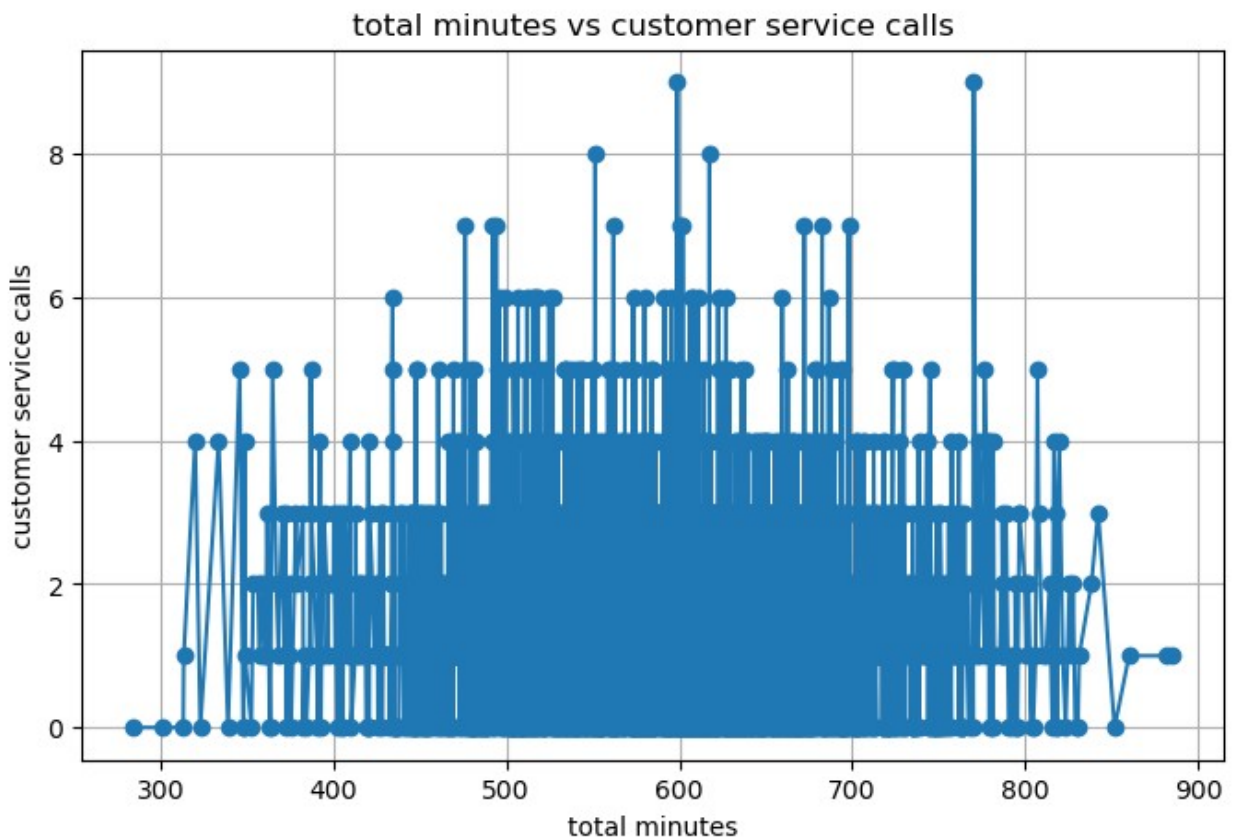
```
#Before plotting, we have to ensure that we have sorted the initial dataframe by total minutes to ensure consistency of the line graph.
```

```
df_sorted = df.sort_values(by = 'total minutes') # This is to ensure that the line graph doesn't end up zig-zag randomly
```

```
plt.figure(figsize=(8,5))  
plt.plot(df_sorted['total minutes'], df_sorted['customer service calls'], marker='o')
```

```
plt.title('total minutes vs customer service calls')  
plt.xlabel('total minutes')  
plt.ylabel('customer service calls')
```

```
plt.grid(True)  
plt.show()
```



From the line graph above, there is no obvious relationship between total minutes and the number of customer service calls. If a customer is to leave the company, it could be attributed to other factors but not really as a result of higher usage clients being subjected to service constraints ending up in customer service calls.

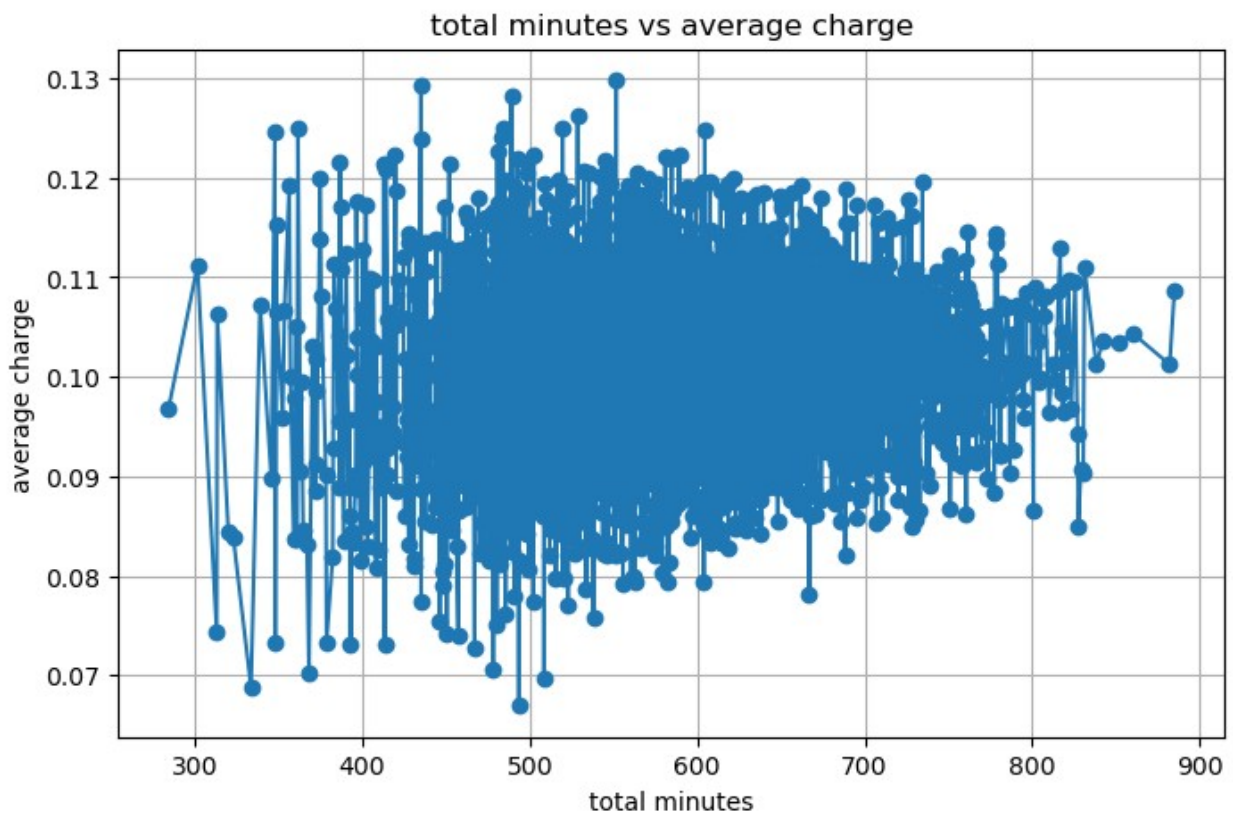
Below is a line graph between total minutes and average charges. This is to show if there is any relationship between total number of minutes and average charges. This is just to see if cost could be the underlying factor to customer churn.

```
#Plotting total minutes against average charge to see if high users end up with larger bills
```

```
plt.figure(figsize=(8,5))  
plt.plot(df_sorted['total minutes'], df_sorted['average charge'],  
marker='o')
```

```
plt.title('total minutes vs average charge')  
plt.xlabel('total minutes')  
plt.ylabel('average charge')
```

```
plt.grid(True)  
plt.show()
```



Again, there is a slight upward linear relationship between total minutes and the average charge, indicating that high usage customers have higher average charges. This isn't sufficient to rely on this as an insight, as the data is scattered and has a lot of outliers. However, there's an interesting aspect of inconsistency in the way the company bills its customers. It is imperative to note that high usage customers should be in a position to get better packages to incentivise their high usage. The inconsistency in charges can be a reason why customers end up leaving but

in a manner that is random and only slightly linear as total minutes go up in relation to average charges.

#Running the dataset for visibility

df.head()

	state	account length	area code	phone number	international	plan	\
0	KS	128	415	382-4657		no	
1	OH	107	415	371-7191		no	
2	NJ	137	415	358-1921		no	
3	OH	84	408	375-9999		yes	
4	OK	75	415	330-6626		yes	

	voice mail plan	number	vmail messages	total day minutes	total day calls	\
0	yes		25	265.1		
110						
1	yes		26	161.6		
123						
2	no		0	243.4		
114						
3	no		0	299.4		
71						
4	no		0	166.7		
113						

	total day charge	...	total night calls	total night charge	\
0	45.07	...	91	11.01	
1	27.47	...	103	11.45	
2	41.38	...	104	7.32	
3	50.90	...	89	8.86	
4	28.34	...	121	8.41	

	total intl minutes	total intl calls	total intl charge	\
0	10.0	3	2.70	
1	13.7	3	3.70	
2	12.2	5	3.29	
3	6.6	7	1.78	
4	10.1	3	2.73	

	customer service calls	churn	total minutes	total charge	average charge
0	1	0	717.2	75.56	
0.105354					
1	1	0	625.2	59.24	
0.094754					
2	0	0	539.4	62.29	
0.115480					
3	2	0	564.8	66.80	
0.118272					

```
4          3      0      512.0      52.09
0.101738
```

```
[5 rows x 24 columns]
```

```
#Running a cross_tab between international plan and churn
```

```
intl_churn_rates = pd.crosstab(df['international plan'], df['churn'])
intl_churn_rates
```

```
#Observations
```

```
#There is a very small portion of the customers who have international plan, and the ones with international plan have a higher churn rate.
```

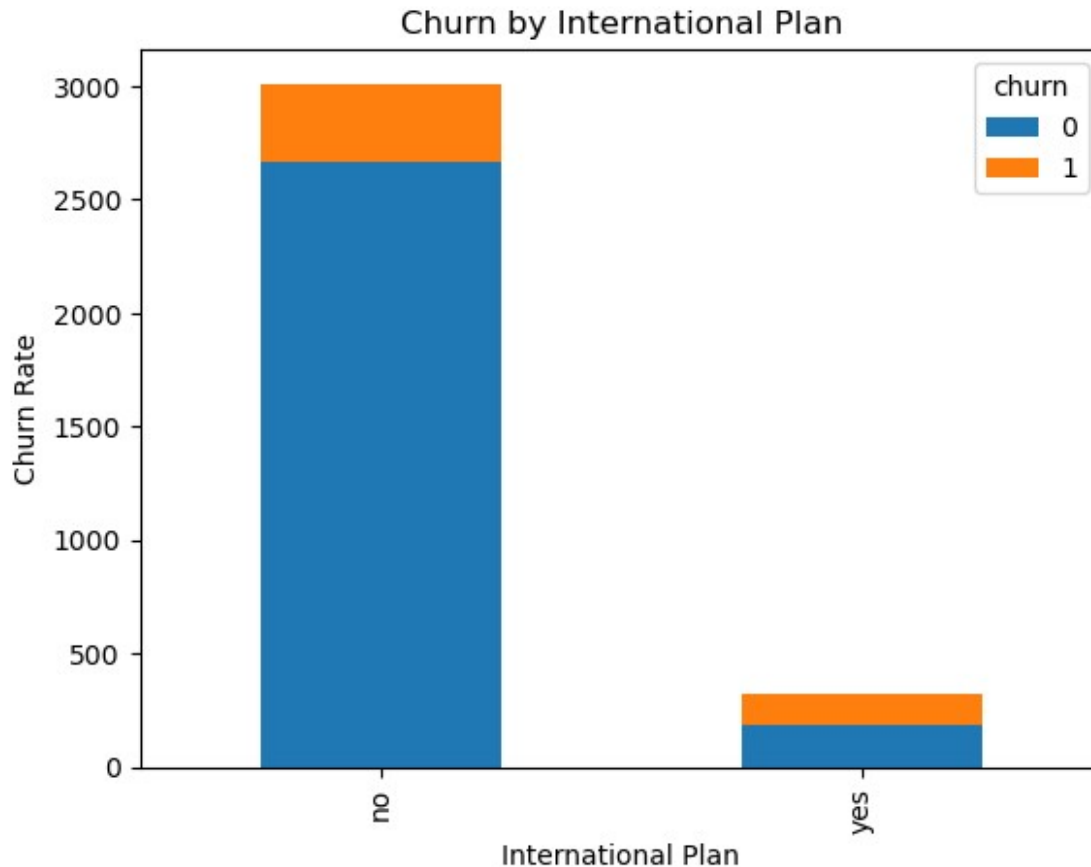
```
#However, numbers wise, the ones with no international plan churn more.
```

```
churn          0      1
international plan
no             2664  346
yes            186  137
```

```
#Comparing how churn rates are between customers with international plans and those who do not have.
```

```
intl_churn_rates = pd.crosstab(df['international plan'], df['churn'])
```

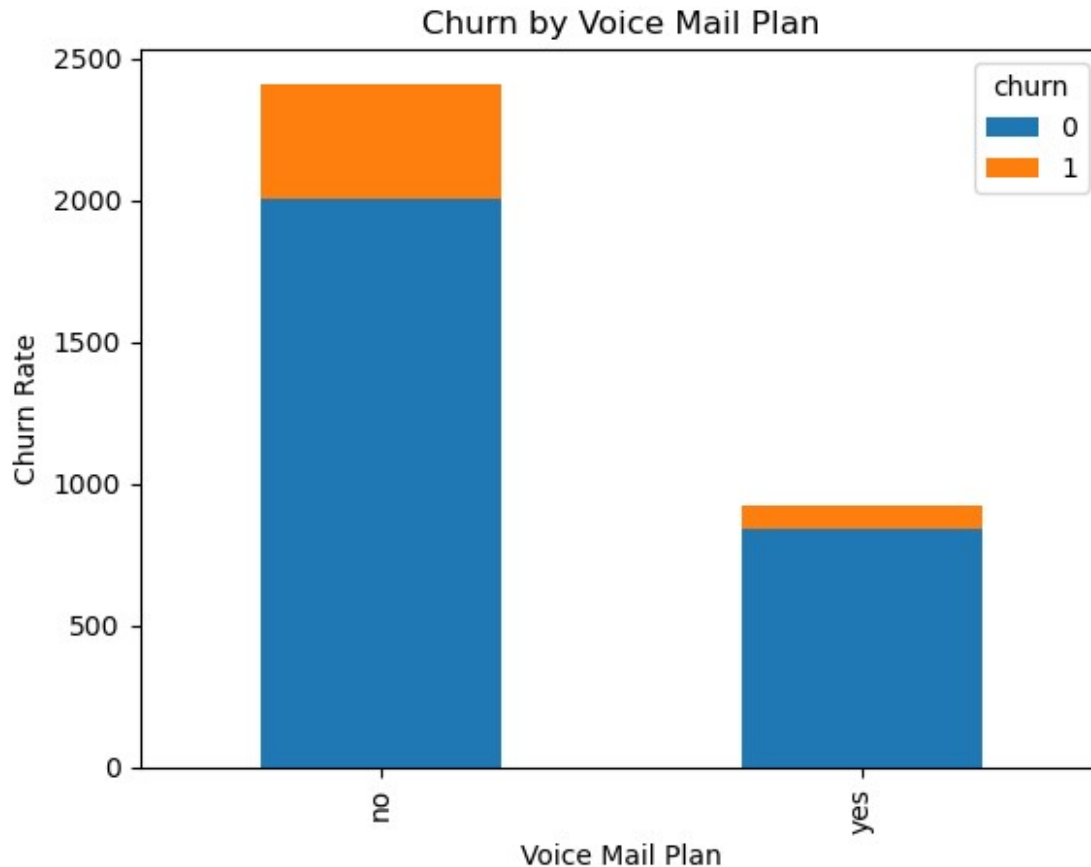
```
intl_churn_rates.plot(kind='bar', stacked =True)
plt.title('Churn by International Plan')
plt.xlabel('International Plan')
plt.ylabel('Churn Rate')
plt.legend(title = 'churn')
plt.show()
```



Based on the visualization above, we can see that from a percentage basis, then the customers with international package shave a higher churn rate. However, looking at it from an aggregate number of clients who have churned, then the customers who do not have international package churn more. This is mainly due to a class imbalance as the customers without international packages are a majority.

#Comparing how churn rates are between customers with voicemail plans and those who do not have.

```
v_mail_churn_rates = pd.crosstab(df['voice mail plan'], df['churn'])  
  
v_mail_churn_rates.plot(kind='bar', stacked =True)  
plt.title('Churn by Voice Mail Plan')  
plt.xlabel('Voice Mail Plan')  
plt.ylabel('Churn Rate')  
plt.legend(title = 'churn')  
plt.show()
```



The above graph shows the relationship between a customer having a voice mail plan and churn rate. The picture is that clients who do not have a voicemail plan tend to churn more than those who do not have voice mail plans. At a preliminary level, it can be said that the company should market its voicemail plans as a way of building loyalty.

#Running the dataset for visibility

`df.head()`

	state	account length	area	code	phone number	international	plan	\
0	KS	128		415	382-4657		no	
1	OH	107		415	371-7191		no	
2	NJ	137		415	358-1921		no	
3	OH	84		408	375-9999		yes	
4	OK	75		415	330-6626		yes	

	voice mail plan	number vmail messages	total day minutes	total day calls	\
0	yes	25	265.1		
110					
1	yes	26	161.6		
123					
2	no	0	243.4		
114					

3	no	0	299.4
71			
4	no	0	166.7
113			

	total day charge	...	total night calls	total night charge	\
0	45.07	...	91	11.01	
1	27.47	...	103	11.45	
2	41.38	...	104	7.32	
3	50.90	...	89	8.86	
4	28.34	...	121	8.41	

	total intl minutes	total intl calls	total intl charge	\
0	10.0	3	2.70	
1	13.7	3	3.70	
2	12.2	5	3.29	
3	6.6	7	1.78	
4	10.1	3	2.73	

	customer service calls	churn	total minutes	total charge	average charge
0	1	0	717.2	75.56	0.105354
1	1	0	625.2	59.24	0.094754
2	0	0	539.4	62.29	0.115480
3	2	0	564.8	66.80	0.118272
4	3	0	512.0	52.09	0.101738

[5 rows x 24 columns]

#Plotting a histogram for the account length against the frequency of Trues in the 'churn' column

#Filtering the rows where churn is 1 i.e True

churn_true = df[df['churn'] == 1]

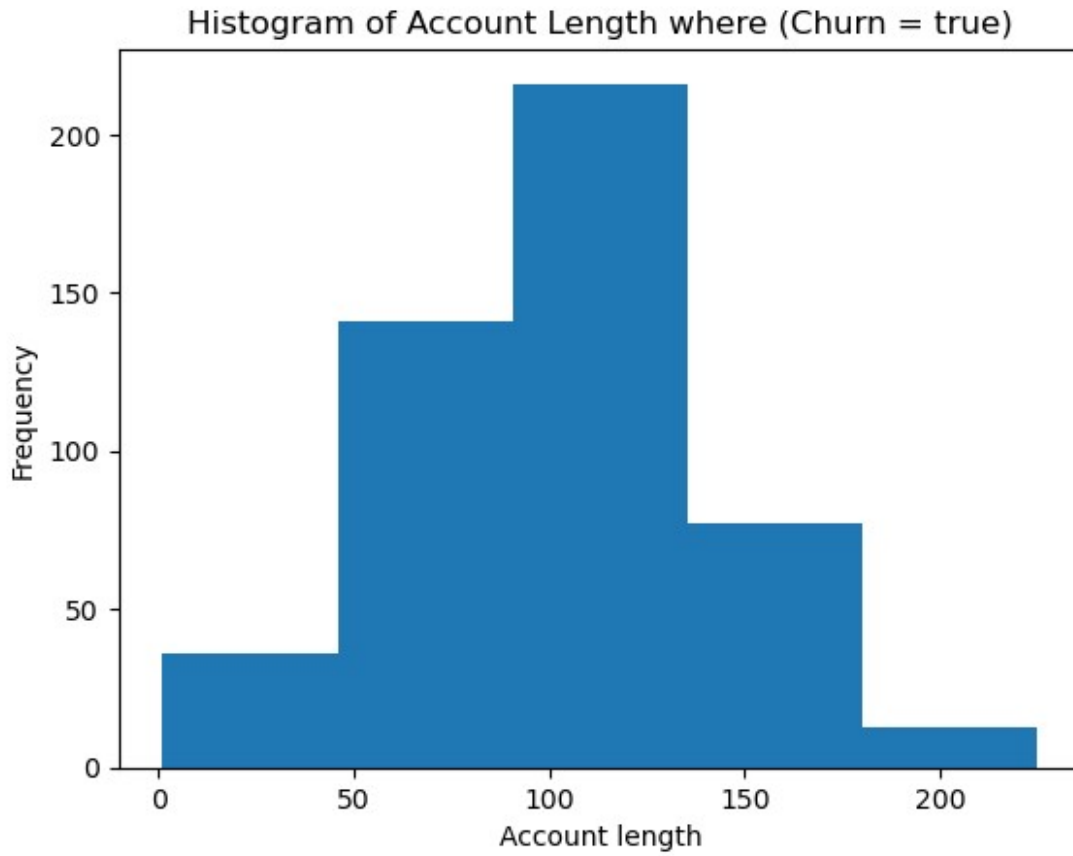
plt.hist(churn_true['account length'], bins= 5)

plt.xlabel('Account length')

plt.ylabel('Frequency')

plt.title('Histogram of Account Length where (Churn = true)')

plt.show()



#The histogram above shows the relationship between the account length and churn rate. We can see that most of the customers leave the company between the 100th and the 150th day of procuring the company's services. The company should then focus mostly on the clients in this and the categories before in order to ensure that they do not churn.

KEY INSIGHTS FROM THE PLOTS ABOVE

1. Customers who have higher average charges per minute tend to churn more as opposed to customers with lower average charges.
2. There's a slight upward relationship between the average charges per minute and total minutes. However, there's quite some inconsistency on how the company charges its clients as the data has a lot of outliers.
3. Customers who have international plans and voicemail plans tend to churn less than clients who do not have any subscription.
4. Customer service calls are not necessarily related to heavy usage customers but their increase leads to customer churn.
5. Customers churn mostly after spending at least 100 days with the company making this the group that the company needs to target more in their efforts to reduce churn rate.

CONSIDERATION OF COLUMNS TO DROP

Some columns are not relevant to the objective of the project and these will be dropped, and the reasons for doing so are listed below:

1. 'State' - The state doesn't really have any sort of relationship to whether a customer leaves the company. Unless non-existent factors like connectivity in some states are brought in. With the prevailing information, there is no relevance as to whether a customer leaves.
2. 'area code' - Again, this has no relevance as to whether a client leaves. This is just for the customer's descriptive purposes.
3. Number of voice mail messages - These are in the control of the client in cases where they do not delete them, cases where they do not receive calls

```
df.head()
```

	state	account length	area code	phone number	international plan \
0	KS	128	415	382-4657	no
1	OH	107	415	371-7191	no
2	NJ	137	415	358-1921	no
3	OH	84	408	375-9999	yes
4	OK	75	415	330-6626	yes

	voice mail plan	number vmail messages	total day minutes	total day calls \
0	yes	25	265.1	110
1	yes	26	161.6	123
2	no	0	243.4	114
3	no	0	299.4	71
4	no	0	166.7	113

	total day charge	...	total night calls	total night charge \
0	45.07	...	91	11.01
1	27.47	...	103	11.45
2	41.38	...	104	7.32
3	50.90	...	89	8.86
4	28.34	...	121	8.41

	total intl minutes	total intl calls	total intl charge \
0	10.0	3	2.70
1	13.7	3	3.70
2	12.2	5	3.29
3	6.6	7	1.78
4	10.1	3	2.73

	customer service calls	churn	total minutes	total charge	average charge
0	1	0	717.2	75.56	0.105354
1	1	0	625.2	59.24	

```

0.094754
2          0          0          539.4          62.29
0.115480
3          2          0          564.8          66.80
0.118272
4          3          0          512.0          52.09
0.101738

```

```
[5 rows x 24 columns]
```

#Dropping unnecessary columns

```

dropped_df = df.drop(columns=['state','area code','number vmail
messages'])
dropped_df.head()

```

```

    account length phone number international plan voice mail plan \
0          128    382-4657                no          yes
1          107    371-7191                no          yes
2          137    358-1921                no          no
3           84    375-9999                yes          no
4           75    330-6626                yes          no

```

```

    total day minutes  total day calls  total day charge  total eve
minutes \
0          265.1          110          45.07
197.4
1          161.6          123          27.47
195.5
2          243.4          114          41.38
121.2
3          299.4          71          50.90
61.9
4          166.7          113          28.34
148.3

```

```

    total eve calls  total eve charge  ...  total night calls \
0           99          16.78  ...          91
1          103          16.62  ...         103
2          110          10.30  ...         104
3           88           5.26  ...          89
4          122          12.61  ...         121

```

```

    total night charge  total intl minutes  total intl calls \
0           11.01          10.0          3
1           11.45          13.7          3
2           7.32          12.2          5
3           8.86           6.6          7
4           8.41          10.1          3

```

	total intl charge	customer service calls	churn	total minutes \
0	2.70	1	0	717.2
1	3.70	1	0	625.2
2	3.29	0	0	539.4
3	1.78	2	0	564.8
4	2.73	3	0	512.0

	total charge	average charge
0	75.56	0.105354
1	59.24	0.094754
2	62.29	0.115480
3	66.80	0.118272
4	52.09	0.101738

[5 rows x 21 columns]

#Converting international plan columnn and voice mail plan columns to 0 ad 1 in stead of True and False for easier modelling.

```
dropped_df['international plan'] =dropped_df['international
plan'].map({'yes':1,'no':0})
dropped_df['voice mail plan'] =dropped_df['voice mail
plan'].map({'yes':1,'no':0})
dropped_df.head()
```

#The reason for this is because machine learning models interepret better binary data in form of 1 and 0 as opposed to having strings of data.

	account length	phone number	international plan	voice mail plan \
0	128	382-4657	0	1
1	107	371-7191	0	1
2	137	358-1921	0	0
3	84	375-9999	1	0
4	75	330-6626	1	0

	total day minutes \	total day calls	total day charge	total eve
0	265.1	110	45.07	197.4
1	161.6	123	27.47	195.5
2	243.4	114	41.38	121.2
3	299.4	71	50.90	61.9
4	166.7	113	28.34	148.3

	total eve calls	total eve charge	...	total night calls \
0	99	16.78	...	91

1	103	16.62	...	103
2	110	10.30	...	104
3	88	5.26	...	89
4	122	12.61	...	121

	total night charge	total intl minutes	total intl calls	\
0	11.01	10.0	3	
1	11.45	13.7	3	
2	7.32	12.2	5	
3	8.86	6.6	7	
4	8.41	10.1	3	

	total intl charge	customer service calls	churn	total minutes	\
0	2.70	1	0	717.2	
1	3.70	1	0	625.2	
2	3.29	0	0	539.4	
3	1.78	2	0	564.8	
4	2.73	3	0	512.0	

	total charge	average charge
0	75.56	0.105354
1	59.24	0.094754
2	62.29	0.115480
3	66.80	0.118272
4	52.09	0.101738

[5 rows x 21 columns]

4. DATA MODELLING AND EVALUATION

```
#Selecting columns that are relevant for our modelling
```

```
cols = ['account length', 'international plan', 'voice mail plan',
'customer service calls', 'churn','total minutes', 'total
charge','average charge']
final_dataset = dropped_df[cols]
final_dataset.head()
```



```
#Colmuns like phone number have not been put as part of the variables
due to being for descriptive purposes
```

```
#Columns about minutes abd charge have been replaced bt the totals and
average columns
```

```
#The number of phone calls made by a customer has been considered to
not have much impact on customer churn
```


	account length	international plan	voice mail plan	\
0	128	0	1	
1	107	0	1	
2	137	0	0	
3	84	1	0	

4	75	1	0
customer service calls	churn	total minutes	total charge
average charge			
0	1	0	717.2
0.105354			75.56
1	1	0	625.2
0.094754			59.24
2	0	0	539.4
0.115480			62.29
3	2	0	564.8
0.118272			66.80
4	3	0	512.0
0.101738			52.09

#Feature selection; This is where we select what is our X and what is our y

#This is to separate our dependent variable 'churn' from the rest of the columns

```
X = final_dataset.drop(columns=['churn'])
y = final_dataset['churn']
```

#First split: Train and validation vs Test

```
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7, stratify=y
)
```

#This is done so that there is separate data for training the model and separate data for validating the model. This helps in avoiding overfitting where

#the model just memorizes instead of learning actual patterns.

#Second split: Train vs validation

```
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.2, random_state=7,
    stratify=y_train_val
)
```

#This is for purposes of avoiding data leakage

DETAIL OF SPLITTING OF DATASET

The dataset was split into training, validation, and test sets. The test set was held out and not used during model training or selection.

After selecting the final model based on validation performance, it was evaluated on the holdout test set to assess generalization to unseen data.

```
#To prevent class imbalance due to the churn clients being fewer than the no churn clients  
y.value_counts(normalize=True)
```

```
churn  
0    0.855086  
1    0.144914  
Name: proportion, dtype: float64
```

```
#Scaling features. This is to ensure that all the numerical values are comparable
```

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_val_scaled = scaler.transform(X_val)  
X_test_scaled = scaler.transform(X_test)
```

```
#Building a dummy model to see if our model can give us a baseline for comparison
```

```
from sklearn.dummy import DummyClassifier  
dummy_model = DummyClassifier(strategy='most_frequent')  
dummy_model.fit(X_train,y_train)  
dummy_model.score(X_val, y_val)
```

```
0.8558052434456929
```

```
from sklearn.metrics import classification_report
```

```
y_dummy = dummy_model.predict(X_val)  
print(classification_report(y_val, y_dummy))
```

```
#The dummy model predicts 0 churns and hence our real model has to outperform this. We will now build our first real model and see the results.
```

	precision	recall	f1-score	support
0	0.86	1.00	0.92	457
1	0.00	0.00	0.00	77
accuracy			0.86	534
macro avg	0.43	0.50	0.46	534
weighted avg	0.73	0.86	0.79	534

```
C:\Users\USER\anaconda3\Lib\site-packages\sklearn\metrics\  
_classification.py:1344: UndefinedMetricWarning: Precision and F-score  
are ill-defined and being set to 0.0 in labels with no predicted  
samples. Use `zero_division` parameter to control this behavior.  
_warn_prf(average, modifier, msg_start, len(result))
```

```
C:\Users\USER\anaconda3\Lib\site-packages\sklearn\metrics\
_classification.py:1344: UndefinedMetricWarning: Precision and F-score
are ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, msg_start, len(result))
C:\Users\USER\anaconda3\Lib\site-packages\sklearn\metrics\
_classification.py:1344: UndefinedMetricWarning: Precision and F-score
are ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, msg_start, len(result))
```

*#Building our first model which is a Logistic Regression model
 #Logistic regression is chosen as the baseline model because it's
 simple, fast, and easily interpretable, making it a good choice as a
 baseline model.*

```
from sklearn.linear_model import LogisticRegression
```

```
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_scaled, y_train)
```

```
LogisticRegression(max_iter=1000)
```

#Evaluation our model to see if it outperforms our dummy model.

```
y_pred = log_model.predict(X_val_scaled)
print(classification_report(y_val, y_pred))
```

	precision	recall	f1-score	support
0	0.88	0.98	0.93	457
1	0.64	0.23	0.34	77
accuracy			0.87	534
macro avg	0.76	0.61	0.64	534
weighted avg	0.85	0.87	0.84	534

The model above outperforms the dummy model as the recall for Churns is 0.23 compared to 0.00 for the dummy model. This is to say that our model predicts 23% of actual churn clients.

The model also has a precision of 0.64 for churn clients compared to a precision of 0.00 in the dummy model. This is to say out of the clients that the model has flagged, 64% actually are churn clients.

Advantages of Logistic Regression

- Simplicity
- Easy of interpretation
- Fast to train

Disadvantages of Logistic Regression

- It assumes a linear relationship between variables and most real time data isn't linear.
- It isn't good in handling mixed data types making it less applicable to real life situations.

The reasons above then makes it inappropriate to leave it at the logistic regression point as there may be limitations on the results obtained as a result of the shortcomings of the model. We will therefore improve to a decision tree due to the accompanying advantages below.

Advantages of Decision Trees

- Ability to handle mixed types of data compared to logistic regression
- Ability to capture nonlinear patterns naturally

```
#Improvement of the baseline model which is the Logistic Regression
#We will do a Decision Tree as our improved model
from sklearn.tree import DecisionTreeClassifier
```

```
d_tree = DecisionTreeClassifier(max_depth=5, random_state=7)
d_tree.fit(X_train, y_train)
```

```
DecisionTreeClassifier(max_depth=5, random_state=7)
```

```
#Making predictions using the decision tree model above
y_pred_tree = d_tree.predict(X_val)
```

```
#Get classification report for the Decision Tree model
```

```
print(classification_report(y_val, y_pred_tree))
```

	precision	recall	f1-score	support
0	0.95	0.99	0.97	457
1	0.95	0.69	0.80	77
accuracy			0.95	534
macro avg	0.95	0.84	0.88	534
weighted avg	0.95	0.95	0.95	534

The model compared to the basic logistic regression model has a recall rate of 69% on the churn customers compared to 0.23; it also has a precision of 95% compared to 64%. The f1 score has also improved from 34% to 80%. This is an indication that it surpasses the basic logistic regression model built above.

As much as the decision tree improved on the logistic regression model, it comes with the limitations below among others. **Limitations of Decision Trees**

- Decision trees are more sensitive to outliers and random fluctuations.
- Decision trees may not be good in capturing complex relationships
- Decisions trees use one model/tree which makes it sensitive.

The limitations above make it necessary that we build another model i.e Random Forest Model, which curbs the aforementioned limitations.

```
#Improving on the decision tree model through a random forest model

from sklearn.ensemble import RandomForestClassifier

rand_forest = RandomForestClassifier(n_estimators=100, random_state=7)
rand_forest.fit(X_train, y_train)

RandomForestClassifier(random_state=7)

#Making predictions using the Random Forest Model Above

y_pred_rand_forest = rand_forest.predict(X_val)

#Running the classification report for the model
print(classification_report(y_val, y_pred_rand_forest))
```

	precision	recall	f1-score	support
0	0.95	1.00	0.97	457
1	0.96	0.69	0.80	77
accuracy			0.95	534
macro avg	0.96	0.84	0.89	534
weighted avg	0.95	0.95	0.95	534

The Random Forest and Decision Tree models have the same precision, f1 score, and recall rates. This is to say that these metrics only are not enough to decide which model is more accurate than the other. However, Random forest is known to be the model that is more suitable for real world situations due to its stability and ability to interpret unseen situations better. For that reason, Random Forest was chosen to be the best model.

5.FINAL MODEL SELECTION

After review of the 3 models, the Random Forest Model was selected to be the final model due to its natural ability to predict real world scenarios. The reason for this is that the evaluation metrics between the Random Forest model and the Decision Tree model were identical and could not form a reliable basis for choosing the better model.

Despite the evaluation metrics being identical between the Random Forest and Decision Tree models, the performance metrics for churn cases were as below:

Accuracy: 0.95 Recall: 0.69 Precision: 0.96

The recall being the priority here, this means that the model will flag 69% churn customers correctly and the company will be able to employ retention strategies leading to a reduced churn rate.

#FINAL MODEL EVALUATION

```
final_model = rand_forest
```

```
y_test_pred= final_model.predict(X_test)
```

```
print(classification_report(y_test, y_test_pred))
```

	precision	recall	f1-score	support
0	0.93	0.99	0.96	570
1	0.93	0.59	0.72	97
accuracy			0.93	667
macro avg	0.93	0.79	0.84	667
weighted avg	0.93	0.93	0.93	667

What has been considered in the classification report of the final model is the churn rate of 59% on the churn customers. This is to say that the model will correctly predict approximately 59% of the customers who are to churn soon. It also has to be considered that the model has a precision of 93% on the churn clients and an accuracy of 93%.

#Feature importance

#This is to ascertain which of the factors that had been considered to be relevant to this research contribute more to the customer churn.

```
feature_importances = pd.Series(  
    final_model.feature_importances_, index=X.columns  
) .sort_values(ascending=False)
```

```
feature_importances.head()
```

```
total charge      0.332860  
total minutes     0.177135  
customer service calls 0.170120  
average charge    0.113995  
account length    0.076440  
dtype: float64
```

6. BUSINESS RECOMMENDATIONS Based on the model insights:

1. The company should target customers with high total charges and employ retention measures like incentives so that they do not end up churning.
2. Clients with high amounts of minutes should also be targeted for retention measures.
3. Clients with high customer service calls also need to be addressed to reduce their churn rate.
4. Clients with higher average charge may need to be given incentivised packages to ensure retention
5. Target to reach out to clients as they approach more than 3 months with the company.

EXPECTED IMPACT

1. Reduced churn rate
2. Improved customer satisfaction

LIMITATIONS

- The dataset may not consider all factors influencing churn like competitor pricing or behavior
- Class imbalance may affect the performance of the model
- The use of historical data which may not be applicable to the future.